

BCC202 - Estruturas de Dados I

Aula 23: Árvores AVL

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Abordagens de pesquisa em Memória Primária

- ▶ Pesquisa Sequencial.
- ▶ Pesquisa Binária.
- ▶ Árvores de Pesquisa:
 - ▶ Árvores Binárias de Pesquisa.
 - ▶ Árvores AVL.
- ▶ Transformação de Chave (*Hashing*):
 - ▶ Listas Encadeadas.
 - ▶ Endereçamento Aberto.
 - ▶ *Hashing* Perfeito.

Conteúdo

Introdução

Árvore AVL

Análise

Aplicações Árvores Binárias de Pesquisa

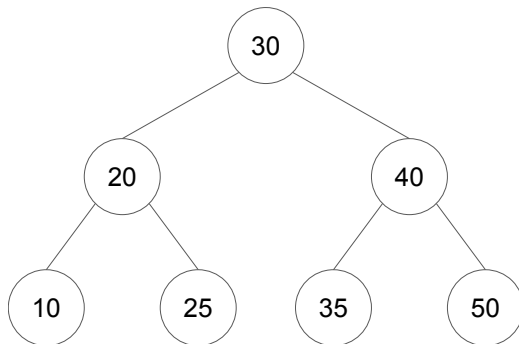
Considerações Finais

Bibliografia

Introdução

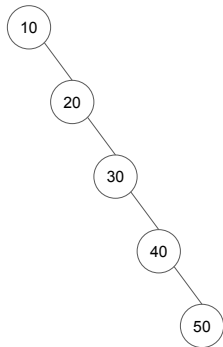
Árvore Binária de Pesquisa:

Inserindo os nós 30, 20, 40, 10, 25, 35 e 50 nesta ordem, teremos:



Árvore Binária de Pesquisa:

Inserindo os nós 10, 20, 30, 40 e 50 nesta ordem, teremos:



Árvores Binárias Balanceadas

- ▶ Existem ordens de inserção de nós que conservam o balanceamento de uma árvore binária.
- ▶ Na prática é impossível prever essa ordem ou até alterá-la.
 - ▶ Algoritmos para balanceamentos.
- ▶ A vantagem de uma árvore balanceada com relação a uma degenerada está em sua eficiência.
- ▶ Por exemplo:
 - ▶ Em uma árvore binária degenerada de 10.000 nós são necessárias, em média, 5.000 comparações (semelhança com arrays ordenados e listas encadeadas).
 - ▶ Numa árvore balanceada com o mesmo número de nós essa média reduz-se a 14 comparações.

Definição de Árvores Balanceadas

- ▶ Uma árvore binária balanceada é aquela na qual, para cada nó, as alturas de suas sub-árvores esquerda e direita diferem de, no máximo, 1.
- ▶ Fator de balanceamento (FB) de um nó é a diferença entre a altura da sub-árvore esquerda em relação à sub-árvore direita.

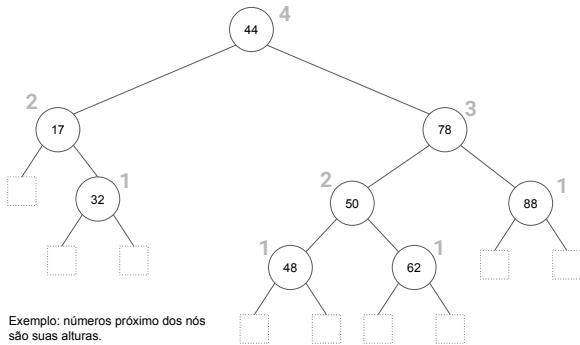
$$FB(p) = altura(\text{sub-árvore esquerda de } p) - altura(\text{sub-árvore direita de } p)$$

- ▶ Em uma árvore binária balanceada todos os FB de todos os nós estão no intervalo $-1 \leq FB \leq 1$

Árvore AVL

Árvore AVL

- ▶ **Árvore binária de busca** tal que, para qualquer nó interno v , *a diferença das alturas dos filhos de v é no máximo 1*.
- ▶ Árvores AVL são balanceadas



AVL

- ▶ Algoritmo de balanceamento de árvores binárias.
- ▶ A origem da denominação AVL vem dos seus dois criadores:
Adel'son-**V**el'skii e **L**andis.
- ▶ Ano de divulgação: 1962.

<https://www.inf.ufsc.br/~aldo.vw/estruturas/simulador/AVL.html>

TAD - Árvores AVL

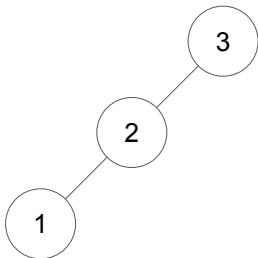
```
1 typedef long Chave;
2
3 typedef struct {
4     Chave chave;
5     /* outros componentes */
6 } Item;
7
8 typedef struct no {
9     Item item;
10    struct no* pEsq;
11    struct no* pDir;
12 } No;
13
14 typedef No* Arvore;
```


Inserção em Árvores AVL

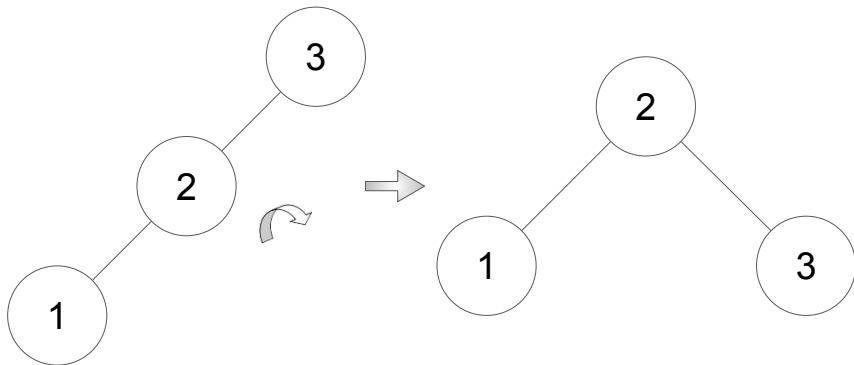
- ▶ Inicialmente inserimos um novo nó na árvore normalmente.
- ▶ A inserção deste pode degenerar a árvore (desbalancear).
- ▶ A restauração do balanceamento é feita por meio de rotações na árvore no nó “**pivô**”.
- ▶ Nó “**pivô**” é aquele que após a inserção possui **Fator de Balanceamento** fora do intervalo.
- ▶ Como resolver? **Rotações!**

AVL - Rotação Simples para a Direita

- ▶ $FB > 1$ (subárvore esquerda maior que subárvore direita).
- ▶ E a subárvore esquerda desta subárvore esquerda é maior que a subárvore direita dela.
- ▶ Então realizar uma rotação simples para a direita.

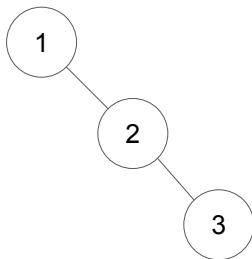


AVL - Rotação Simples para a Direita

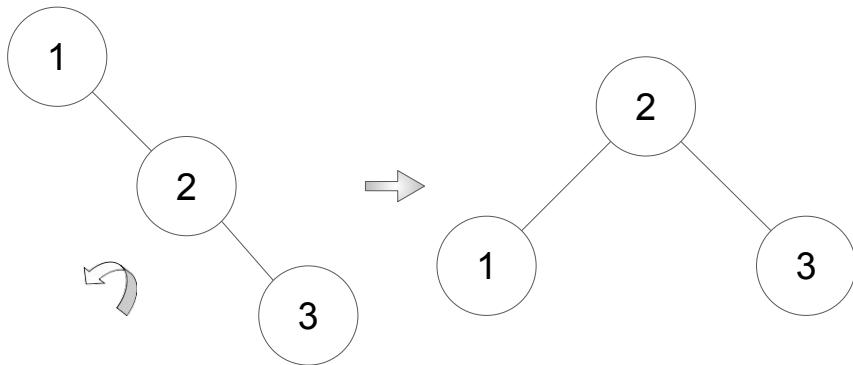


AVL - Rotação Simples para a Esquerda

- ▶ $FB < -1$ (subárvore esquerda menor que subárvore direita).
- ▶ E a subárvore direita desta subárvore direita é maior que a subárvore esquerda dela.
- ▶ Então realizar uma rotação simples para a esquerda.

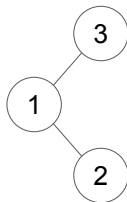


AVL - Rotação Simples para a Esquerda

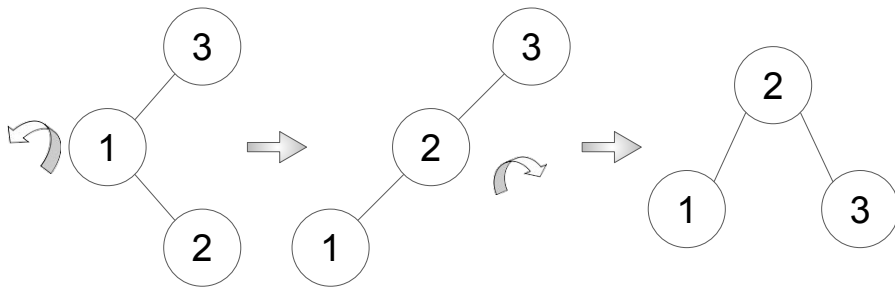


AVL - Rotação Dupla para a Direita

- ▶ $FB > 1$ (subárvore esquerda maior que subárvore direita).
- ▶ E a subárvore esquerda desta subárvore esquerda é menor ou igual que a subárvore direita dela.
- ▶ Então realizar uma rotação dupla para a direita.

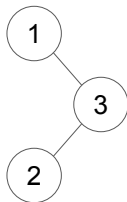


AVL - Rotação Dupla para a Direita

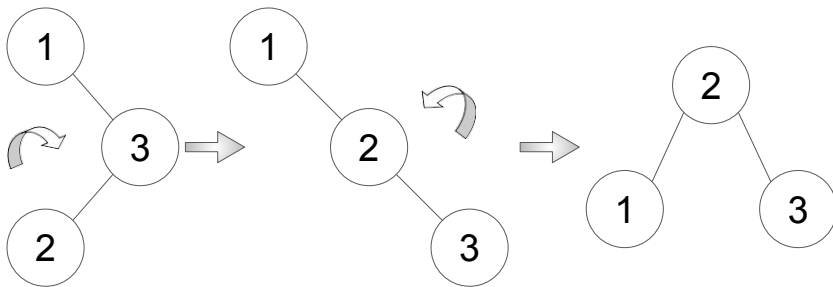


AVL - Rotação Dupla para a Esquerda

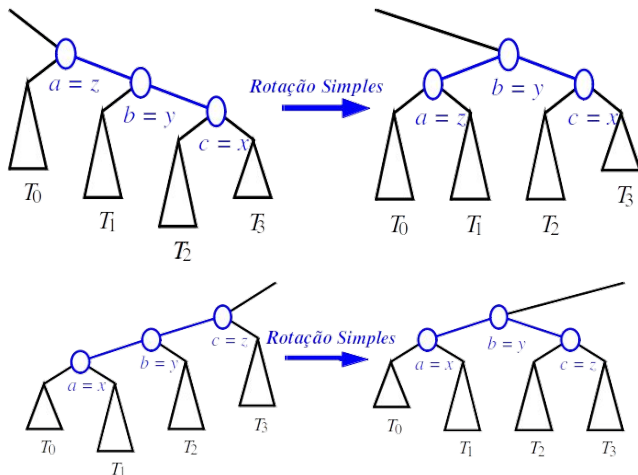
- ▶ $FB < -1$ (subárvore esquerda menor que subárvore direita).
- ▶ E a subárvore direita desta subárvore direita é menor que a subárvore esquerda dela.
- ▶ Então realizar uma rotação dupla para a esquerda.



AVL - Rotação Dupla para a Esquerda



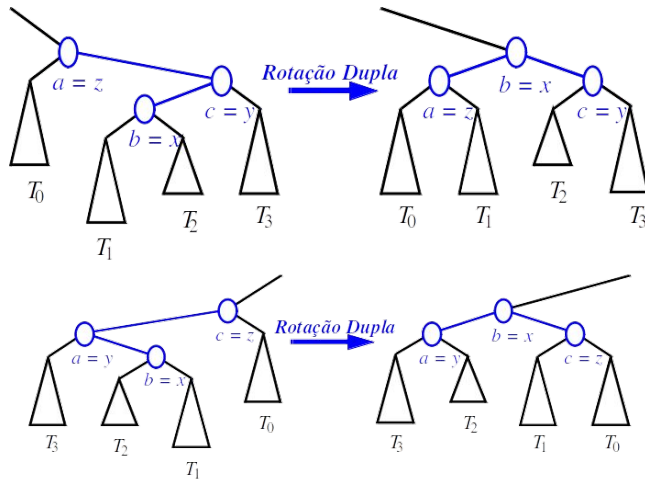
Exemplos de Rotação Simples



Rotação Simples

```
1 void RSE(No** ppRaiz) {
2     No *pAux;
3     pAux = (*ppRaiz)->pDir;
4     (*ppRaiz)->pDir = pAux->pEsq;
5     pAux->pEsq = (*ppRaiz);
6     (*ppRaiz) = pAux;
7 }
8
9 void RSD(No** ppRaiz) {
10     No *pAux;
11     pAux = (*ppRaiz)->pEsq;
12     (*ppRaiz)->pEsq = pAux->pDir;
13     pAux->pDir = (*ppRaiz);
14     (*ppRaiz) = pAux;
15 }
```

Exemplos de Rotação Dupla



Rotação Dupla

```
1 bool BalancaEsquerda(No** ppRaiz) {
2     int fbe = FB( (*ppRaiz)->pEsq );
3     if ( fbe >= 0 )
4         RSD(ppRaiz);
5     else {
6         /* Rotação Dupla Direita */
7         RSE( &((*ppRaiz)->pEsq) );
8         RSD( ppRaiz ); /* &(*ppRaiz) */
9     }
10    return true;
11 }
```

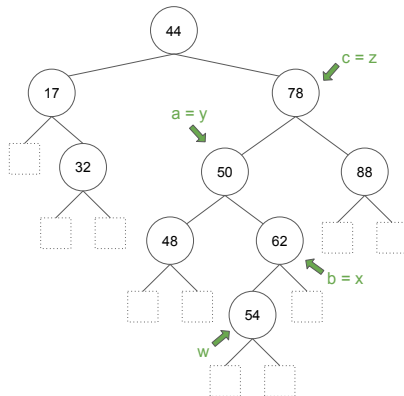
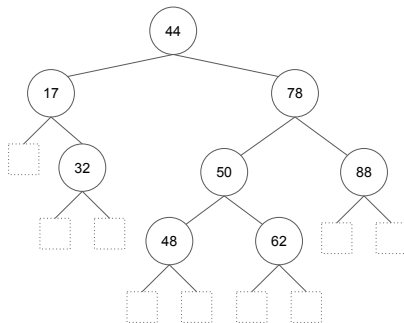
```
1 int BalancaDireita(No** ppRaiz) {
2     int fbd = FB( (*ppRaiz)->pDir);
3     if ( fbd <= 0 )
4         RSE (ppRaiz);
5     else {
6         /* Rotação Dupla Esquerda */
7         RSD( &((*ppRaiz)->pDir) );
8         RSE( ppRaiz ); /* &(*ppRaiz) */
9     }
10    return true;
11 }
```

Balanceamento

```
1 bool Balanceamento(No** ppRaiz) {  
2     int fb = FB(*ppRaiz);  
3     if ( fb > 1)  
4         return BalancaEsquerda(ppRaiz);  
5     else if (fb < -1 )  
6         return BalancaDireita(ppRaiz);  
7     return false;  
8 }
```

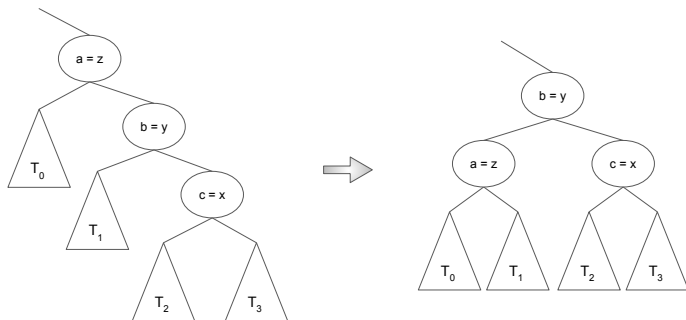
Inserção em uma Árvore AVL

- ▶ Inserção como em uma árvore binária de pesquisa.
- ▶ Sempre feita expandindo um nó externo.



Reestruturação Trinodo

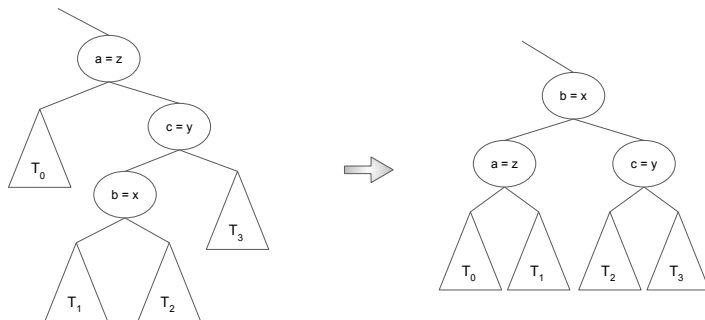
- ▶ Os nós x , y , z (*filho*, *pai*, *avô*) renomeados como a , b , c (percurso inter-fixado).
- ▶ Rotações levam b para o topo.



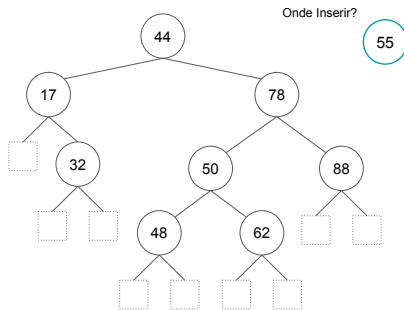
Outros dois casos são simétricos

Reestruturação Trinodo

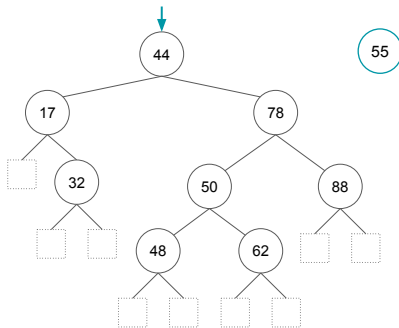
- ▶ Os nós x , y , z (*filho*, *pai*, *avô*) renomeados como a , b , c (percurso inter-fixado).
- ▶ Rotações levam b para o topo.



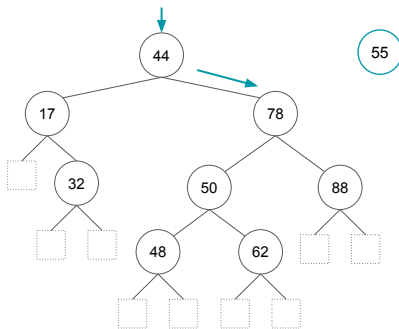
Exemplo de Inserção de um nó



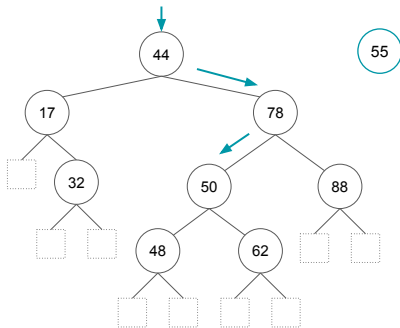
Exemplo de Inserção de um nó



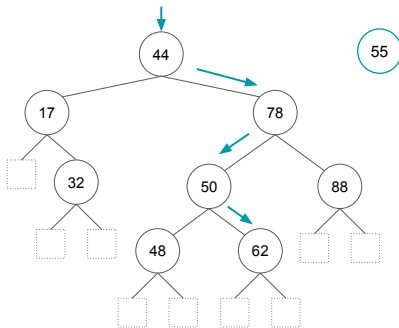
Exemplo de Inserção de um nó



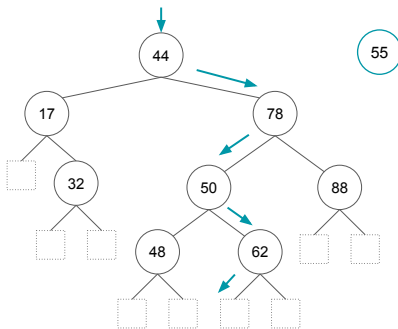
Exemplo de Inserção de um nó



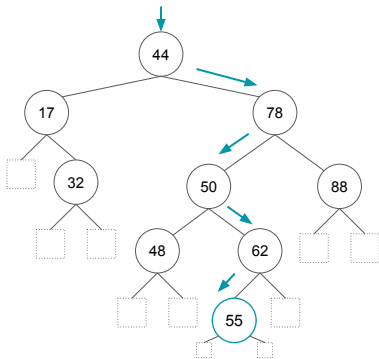
Exemplo de Inserção de um nó



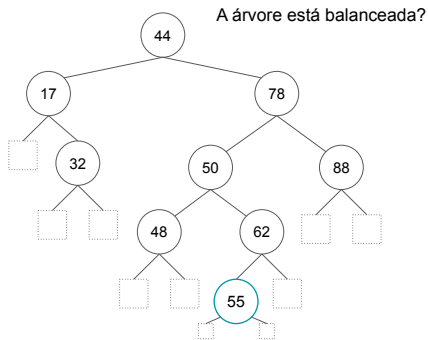
Exemplo de Inserção de um nó



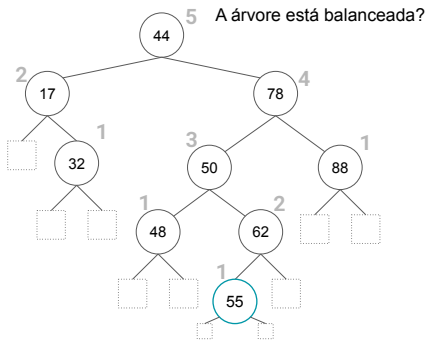
Exemplo de Inserção de um nó



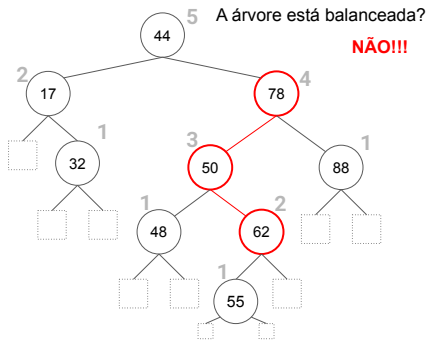
Exemplo de Inserção de um nó



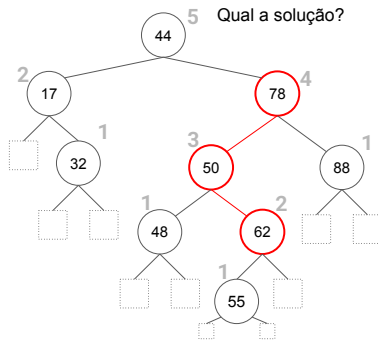
Exemplo de Inserção de um nó



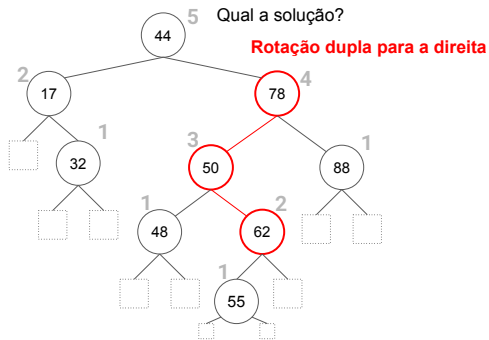
Exemplo de Inserção de um nó



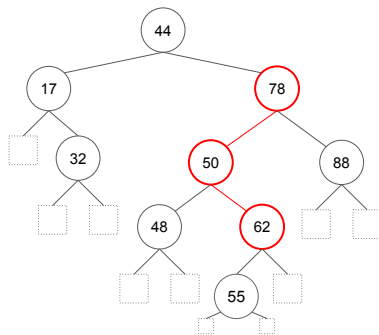
Exemplo de Inserção de um nó



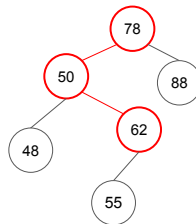
Exemplo de Inserção de um nó



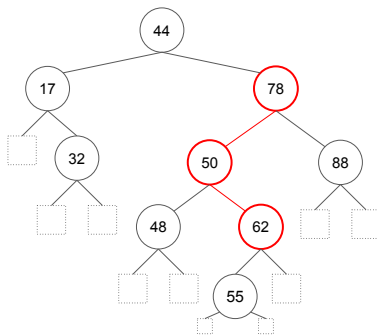
Exemplo de Inserção de um nó



Rotação dupla para a direita

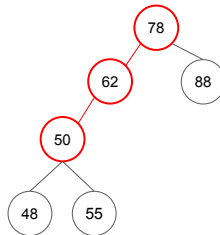


Exemplo de Inserção de um nó

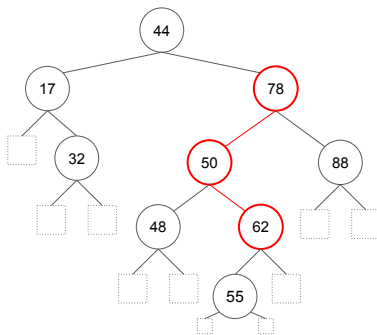


Rotação dupla para a direita

Rotação simples para esquerda

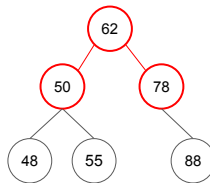


Exemplo de Inserção de um nó

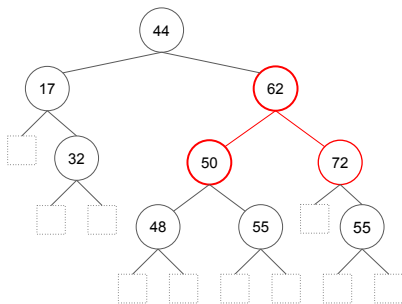


Rotação dupla para a direita

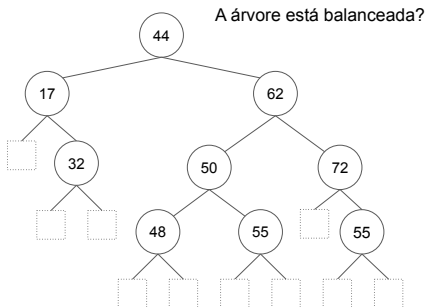
Rotação simples para direita



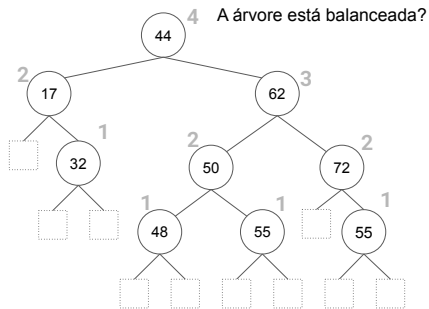
Exemplo de Inserção de um nó



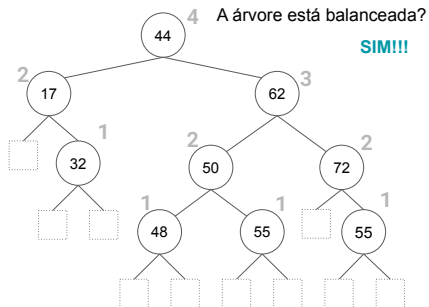
Exemplo de Inserção de um nó



Exemplo de Inserção de um nó



Exemplo de Inserção de um nó



Inserção de um nó em uma Árvore AVL

```

1 bool Insere(No** ppRaiz, Item* x) {
2     if (*ppRaiz == NULL) {
3         *ppRaiz = NoCria(x);
4         return true;
5     } else if ((*ppRaiz)->item.chave > x->chave) {
6         if (Insere(&(*ppRaiz)->pEsq,x)) {
7             if (Balanceamento(ppRaiz))
8                 return false;
9             else
10                return true;
11        }
12    } else if ((*ppRaiz)->item.chave < x->chave) {
13        if (Insere(&(*ppRaiz)->pDir,x)) {
14            if (Balanceamento(ppRaiz))
15                return false;
16            else
17                return true;
18        }
19    }
20    return false; /* valor jah presente */
21 }

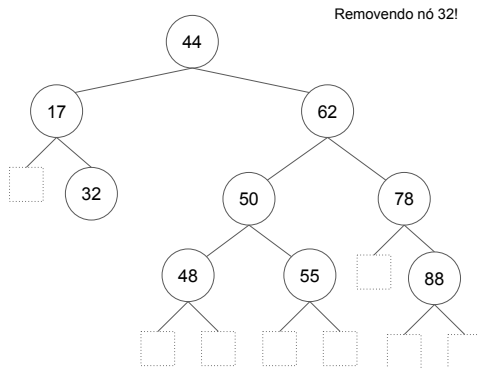
```

Análise Implementação da Inserção

- ▶ Cálculo de fatores de balanceamento:
 - ▶ Custo: $O(\log n)$??
- ▶ Como melhorar?
 - ▶ Cada nó:
 - ▶ Fator de balanceamento.
 - ▶ Profundidade x Altura.
 - ▶ Problema: atualizar dados durante rotações.

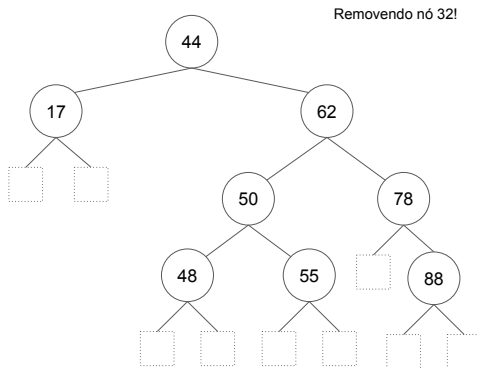
Remoção em uma Árvore AVL

- ▶ Remoção começa como em uma árvore binária de busca.
⇒ pode causar desbalanceamento.



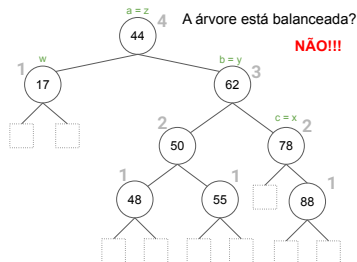
Remoção em uma Árvore AVL

- ▶ Remoção começa como em uma árvore binária de busca.
⇒ pode causar desbalanceamento.

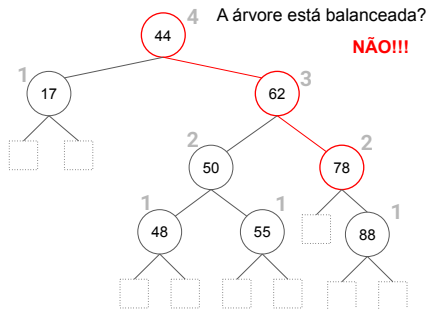


Rebalanceamento Após uma Remoção

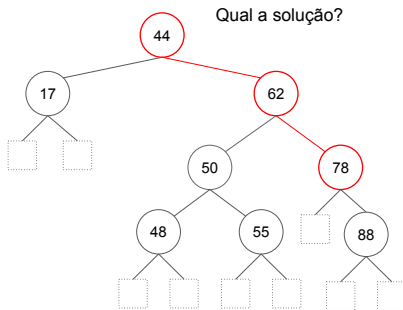
- ▶ Seja **z** o **primeiro nó desbalanceado** encontrado acima de **w**.
- ▶ Seja **y** o filho de **z** com maior altura, e **x** o filho de **y** com maior altura.
- ▶ Executar *Balanceamento*(**x**) para rebalancear **z**.
- ▶ Pode ocorrer desbalanceamento de outro nó acima $\Rightarrow$ continuar verificação de balanceamento até à raiz.



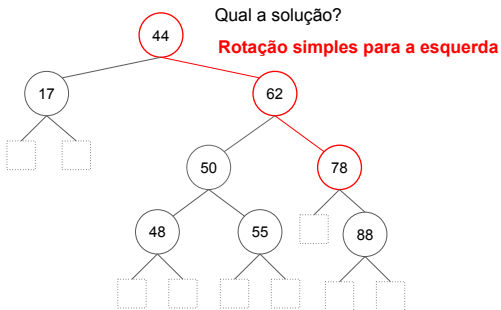
Exemplo de Remoção



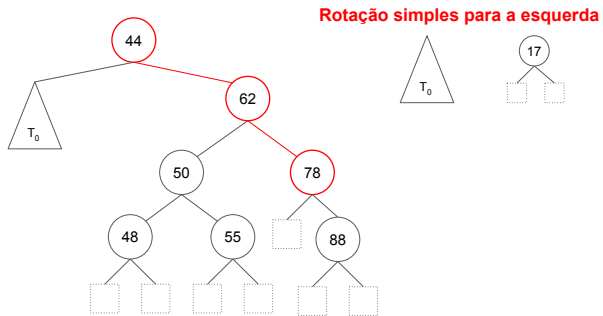
Exemplo de Remoção



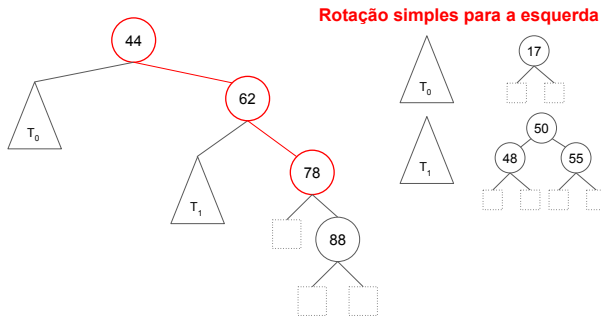
Exemplo de Remoção



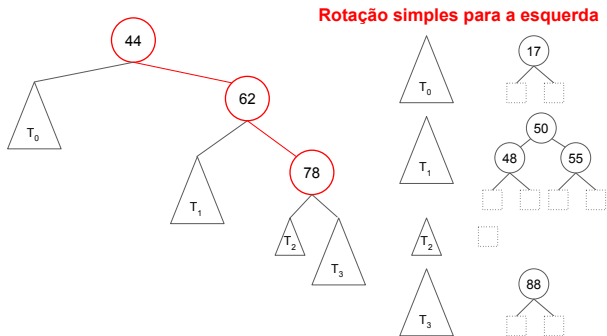
Exemplo de Remoção



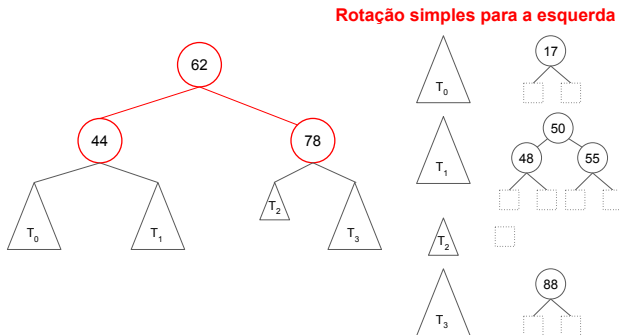
Exemplo de Remoção



Exemplo de Remoção

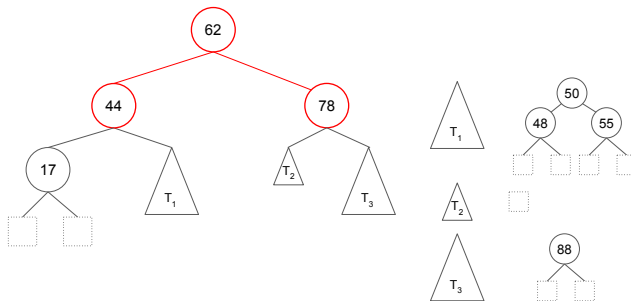


Exemplo de Remoção



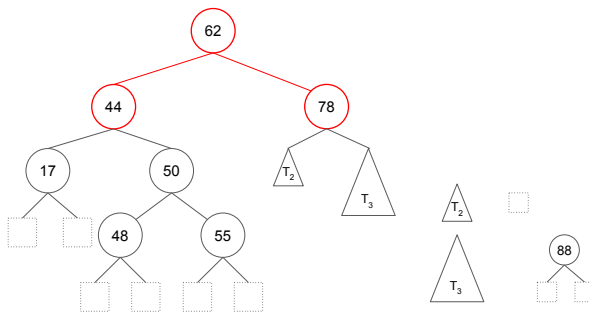
Exemplo de Remoção

Rotação simples para a esquerda



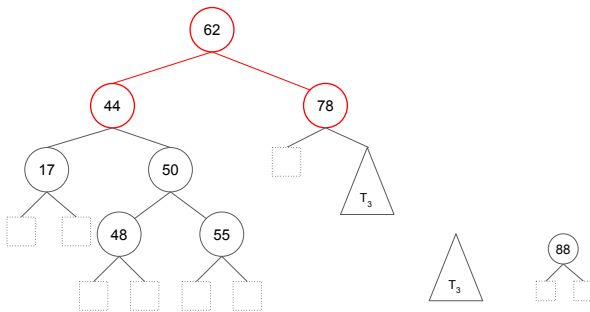
Exemplo de Remoção

Rotação simples para a esquerda



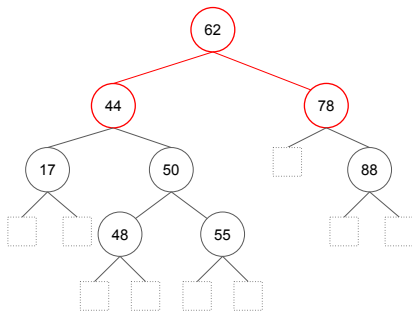
Exemplo de Remoção

Rotação simples para a esquerda



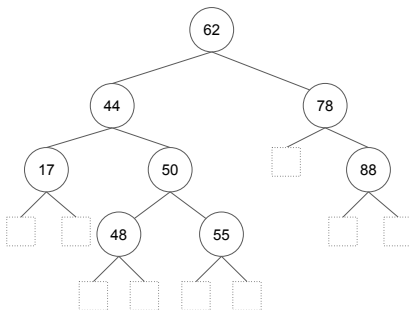
Exemplo de Remoção

Rotação simples para a esquerda



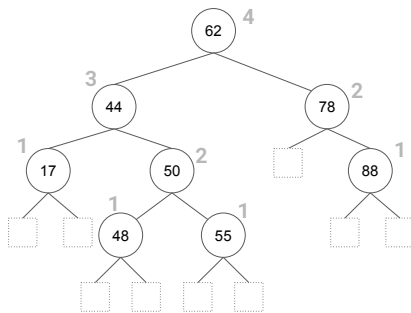
Exemplo de Remoção

A árvore está balanceada?



Exemplo de Remoção

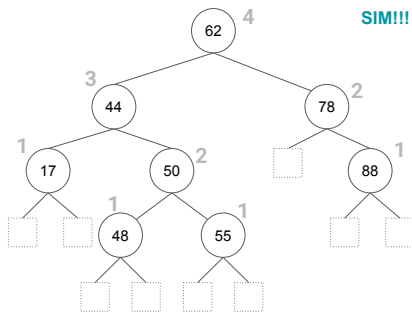
A árvore está balanceada?



Exemplo de Remoção

A árvore está balanceada?

SIM!!!



```

1 bool Remove (No** ppRaiz, Item* pX) {
2     if (*ppRaiz == NULL)
3         return false;
4     else if ((*ppRaiz)->item.chave == pX->chave) {
5         *pX = (*ppRaiz)->item;
6         Antecessor(ppRaiz, &((*ppRaiz)->pEsq));
7         Balanceamento(ppRaiz);
8         return true;
9     } else if ((*ppRaiz)->item.chave > pX->chave) {
10        if (Remove(&(*ppRaiz)->pEsq, pX)) {
11            Balanceamento(ppRaiz);
12            return true;
13        }
14    } else {
15        if (Remove(&(*ppRaiz)->pDir, pX)) {
16            Balanceamento(ppRaiz);
17            return true;
18        }
19    }
20    return false;
21 }

```

Análise

Complexidade de Tempo para Árvores AVL

- ▶ Uma única reestruturação é $O(1)$.
 - ▶ Usando uma árvore binária implementada com estrutura ligada.
- ▶ Pesquisa é $O(\log n)$.
 - ▶ Altura de árvore é $O(\log n)$, não necessita reestruturação.
- ▶ Inserir é $O(\log n)$.
 - ▶ Busca inicial é $O(\log n)$.
 - ▶ Reestruturação para manter balanceamento é $O(\log n)$.
- ▶ Remove é $O(\log n)$.
 - ▶ Busca inicial é $O(\log n)$.
 - ▶ Reestruturação para manter balanceamento é $O(\log n)$.

Verifica se uma árvore é AVL

```
1 bool EhArvoreAvl(No* pRaiz) {  
2     int fb;  
3     if (pRaiz == NULL)  
4         return true;  
5     if (!EhArvoreArvl(pRaiz->pEsq))  
6         return false;  
7     if (!EhArvoreArvl(pRaiz->pDir))  
8         return false;  
9     fb = FB (pRaiz);  
10    if ((fb > 1) || (fb < -1))  
11        return false;  
12    else  
13        return true;  
14 }
```

Aplicações Árvores Binárias de Pesquisa

Aplicações

- ▶ Para que servem as Árvores Binárias?
- ▶ Exemplos de aplicações:
 - ▶ Redes de Comunicação de Dados:
 - ▶ Envio de pacotes ordenados e/ou redundantes.
 - ▶ Codificação de Huffman.
 - ▶ Compressão e Descompressão de arquivos.

Redes de Comunicação

- ▶ A maioria dos protocolos de comunicação fragmenta as mensagens em pacotes que são numerados e enviados através da rede.
- ▶ Não há garantia da chegada em ordem dos pacotes.
- ▶ Perdas de pacotes geram novos envios e estes podem causar duplicatas dos mesmos

Reconstrução da Mensagem

- ▶ Como reconstruir a mensagem corretamente?
 - ▶ Descartar os pacotes repetidos.
 - ▶ Ordenar os pacotes.
- ▶ Como implementar tal algoritmo?
 - ▶ Utilizando Árvores Binárias

Redes de Comunicação - Algoritmo

- ▶ O primeiro pacote é colocado na raiz da árvore. Cada pacote sucessivo é comparado com o da raiz.
- ▶ Se for igual, descarta-se a réplica. Se for menor ou maior, percorre-se os lados esquerdo ou direito da árvore.
- ▶ Sub-árvore vazia implica inserção do novo pacote.
- ▶ Sub-árvore não vazia implica comparação dos pacotes com a mesma.

Redes de Comunicação - Problemas resolvidos?

Problema da ordenação.

- ▶ A ordenação dos pacotes pode ser feita trivialmente com apenas uma chamada ao método *inOrder()* da árvore binária.

Problema da redundância.

- ▶ Solucionado com o algoritmo de inserção na árvore, visto que o pacote, antes de ser inserido, é comparado com os demais que já se encontram na árvore binária.

Códigos e Caracteres

- ▶ Caracteres são letras, números e símbolos.
- ▶ Códigos são sequências de bits que podem representar de maneira ÚNICA um caractere.
- ▶ **b** bits para representar **c** caracteres: $c = 2^b$.

ASCII (7 bits) $2^7 = 128$ caracteres	Extended ASCII (8 bits) $2^8 = 256$ caracteres
--	---

Codificação de Huffman - Exemplo

- Considere o arquivo com o seguinte texto:

AAAAAAAAAABBBBBBBBCCCCCDDDDDEE

- ▶ Frequências: $A = 10$; $B = 8$; $C = 6$; $D = 5$; $E = 2$.
- ▶ Construção da Árvore Binária.
- ▶ Comparação do número de bits
 - ▶ Tamanho Fixo (8 bits) $\Rightarrow$ Total = 248 bits.
 - ▶ Tamanho Variável $\Rightarrow$ Total = 69 bits

Codificação de Huffman - Descompressão

- ▶ Regeneração da árvore binária através da tabela de caracteres e códigos.
- ▶ A cadeia de bits é percorrida e, à medida que uma sub-cadeia é encontrada na tabela de caracteres e códigos, a mesma é substituída pelo caractere correspondente.

- ▶ As árvores binárias são uma das estruturas de dados mais importantes devido a grande aplicabilidade das mesmas.
- ▶ A maioria dos algoritmos das árvores binárias são de simples entendimento, facilitando sobremaneira o desenvolvimento de sistemas.

BCC202 - Estruturas de Dados I

Exercício

- ▶ Dada a sequência de números: 10, 20, 5, 8, 12, 22, 23, 24, 11, 13, 18.
- ▶ Mostre (desenhe) uma árvore AVL após a inserção de cada um dos elementos acima.
- ▶ Mostre como ficará a árvore criada após a remoção dos seguintes elementos na seguinte ordem: 22, 11, 5, 10.